



Data Structures

(Course Code: 25B11CI318)

juit

विद्या तत्त्व ज्योतिसमः

Priority Queue Using Binary Heap

Mr. Sandeep Kumar Patel

Assistant Professor

Jaypee University of Information Technology

Waknaghat, Solan, HP-173234

What is a Priority Queue?

Definition

A **priority queue** stores elements with associated priorities and provides access to the element with the highest priority under a chosen ordering.

- A max-priority queue returns the largest priority.
- A min-priority queue returns the smallest priority.
- Removal depends on priority, rather than arrival time.

Equal Priorities

A binary heap does not inherently preserve arrival order among equal priorities. Add a sequence number if FIFO tie-breaking is required.

Ordinary Queue vs Priority Queue

Ordinary Queue

First in, first out.

Arrival order:

A, B, C

Removal order:

A, B, C

Max-Priority Queue

Largest priority first.

Priorities:

A : 2, B : 5, C : 3

Removal order:

B, C, A

The priority queue is an **abstract data type**; a binary heap is one way to implement it.

Priority Queue Operations

Operation	Purpose
Insert	Add an element with its priority
Peek-Max / Peek-Min	Read the extreme element without removing it
Extract-Max / Extract-Min	Remove and return the extreme element
Increase-Key / Decrease-Key	Change a priority and restore heap order
Is-Empty	Test whether the queue contains any elements

Convention

The examples use a **max-heap** and treat each key as its priority. Reverse comparisons for a min-heap.

What is a Binary Heap?

Definition

A **binary heap** is a complete binary tree satisfying a heap-order property.

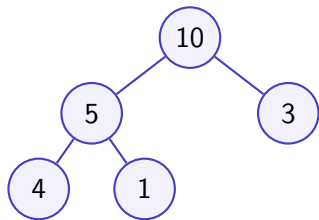
It combines two independent requirements:

- ① **Shape:** all levels before the last are full, and the last level fills from left to right.
- ② **Order:** every parent has the required relationship with each of its children.

Important

A correctly ordered tree is not a binary heap unless its shape is also complete.

Max-Heap



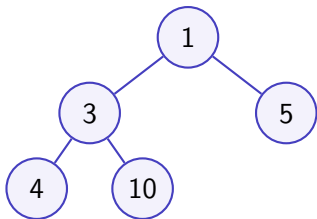
Max-Heap Property

For every non-root node i :

$$A[\text{parent}(i)] \geq A[i]$$

- Every parent dominates its children.
- The root contains a maximum key.
- Equal keys are allowed.

Min-Heap



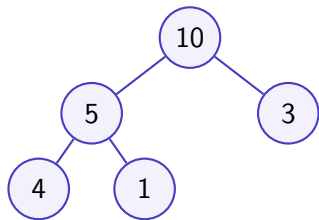
Min-Heap Property

For every non-root node i :

$$A[\text{parent}(i)] \leq A[i]$$

- Every parent is no greater than its children.
- The root contains a minimum key.
- Siblings need not be ordered.

A Heap is Not a Binary Search Tree



- A heap orders parents relative to children.
- A BST orders entire left and right subtrees.
- A heap does not require left keys to be smaller than right keys.

Consequence

Inorder traversal of this heap is 4, 5, 1, 10, 3, which is not sorted.
Searching for an arbitrary key can require $O(N)$ time.

Array Representation

A complete tree fits into consecutive array positions.

For **zero-based indexing**:

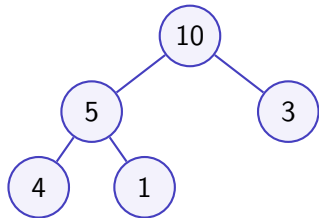
$$\text{Left}(i) = 2i + 1$$

$$\text{Right}(i) = 2i + 2$$

$$\text{Parent}(i) = \left\lfloor \frac{i-1}{2} \right\rfloor, \quad i > 0$$

A child exists only if its index is less than the current heap size.

Tree and Array: Same Heap



0	1	2	3	4
10	5	3	4	1

Example: Index 1

The key is 5.

$$\text{Left}(1) = 3, \quad A[3] = 4$$

$$\text{Right}(1) = 4, \quad A[4] = 1$$

Height, Internal Nodes and Leaves

For a non-empty heap with N nodes:

$$h = \lfloor \log_2 N \rfloor$$

Under zero-based indexing:

- Last internal-node index:

$$\lfloor N/2 \rfloor - 1$$

- Leaf indices:

$$\lfloor N/2 \rfloor, \dots, N - 1$$

For $N = 1$, there are no internal nodes and the root is a leaf.

Maintaining the Heap Property

Sift Up

Used when a key may be too large for its parent.

Compare with the parent and swap upward.

Typical use: insertion into a max-heap.

Sift Down

Used when a key may be too small for its children.

Compare with the larger child and swap downward.

Typical use: extract-max.

At most $O(\log N)$ levels

Insertion into a Max-Heap

- 1 Append the new key at the next free array position.
- 2 This preserves the complete-tree shape.
- 3 Compare the new key with its parent.
- 4 If it is larger, swap upward.
- 5 Stop at the root or when heap order holds.

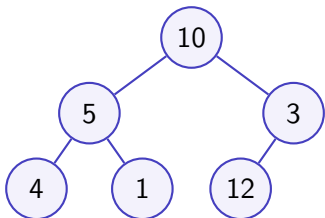
Why Only One Path?

All old parent–child relationships remain valid. Only the new key's ancestor path can require repair.

Insertion Example: Append 12

Initial heap:

[10, 5, 3, 4, 1]



After Appending

[10, 5, 3, 4, 1, 12]

New index: 5

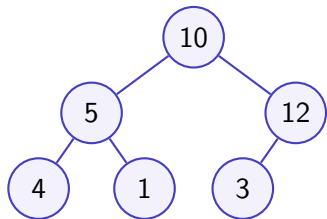
Parent index:

$$\lfloor (5 - 1) / 2 \rfloor = 2$$

Since $12 > 3$, swap.

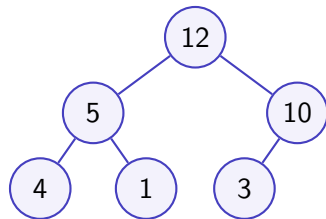
Insertion Example: Sift Up

After the first swap



[10, 5, 12, 4, 1, 3]

After the second swap



[12, 5, 10, 4, 1, 3]

The new key reaches the root. Heap order is restored.

Insertion Algorithm

```
1 MAX-HEAP-INSERT(A, key)
2   i = heapSize
3   A[i] = key
4   heapSize = heapSize + 1
5
6   while i > 0
7       p = floor((i - 1) / 2)
8       if A[p] >= A[i]
9           break
10      SWAP(A[p], A[i])
11      i = p
```

Cost

$O(\log N)$ worst-case time and $O(1)$ auxiliary space, assuming sufficient array capacity.

Accessing the Maximum

The maximum key is at the root:

$$\text{PeekMax}(A) = A[0]$$

- Check that the heap is non-empty.
- Return the root value.
- Do not change the heap.

$$\text{Time} = \Theta(1)$$

Peek vs Extract

Peek only reads the maximum. Extract removes it and restores heap order.

Extract-Max

- ① Save the root value.
- ② Move the last key to the root.
- ③ Reduce the heap size by one.
- ④ Sift the replacement key downward.
- ⑤ Return the saved maximum.

Choice of Child

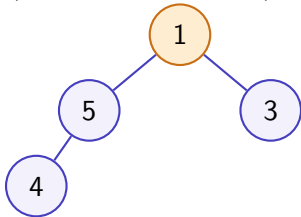
Swap with the **larger child**. Swapping with the smaller child may leave the parent smaller than its other child.

Extract-Max: Move the Last Key

Initial max-heap:

[10, 5, 3, 4, 1]

Remove 10, move 1 to the root, and reduce the size.

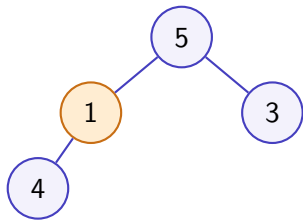


[1, 5, 3, 4]

- Shape is still complete.
- Root order is violated.
- Larger child: 5.

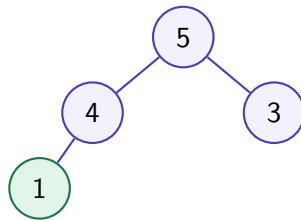
Extract-Max: Restore Heap Order

Swap 1 with 5



[5, 1, 3, 4]

Swap 1 with 4



[5, 4, 3, 1]

Extract-Max Algorithm

```
1 EXTRACT-MAX(A)
2   if heapSize == 0
3       report underflow
4
5   maximum = A[0]
6   A[0] = A[heapSize - 1]
7   heapSize = heapSize - 1
8
9   if heapSize > 0
10       SIFT-DOWN(A, heapSize, 0)
11
12   return maximum
```

Worst-case time = $O(\log N)$

Sift Down: Preconditions

Required Condition

The left and right subtrees of index i must already be max-heaps.
Only the key at i may violate heap order.

- 1 Find the largest among the current key and its existing children.
- 2 If the current key is largest, stop.
- 3 Otherwise swap with the larger child.
- 4 Continue from that child's position.

Important

One max-heapify call at the root does not necessarily turn an arbitrary array into a heap.

Max- heapify Algorithm

```
1 SIFT-DOWN(A, n, i)
2   while 2*i + 1 < n
3       child = 2*i + 1
4
5       if child + 1 < n and A[child+1] > A[child]
6           child = child + 1
7
8       if A[i] >= A[child]
9           break
10
11      SWAP(A[i], A[child])
12      i = child
```

Meaning of n

Only indices $0, \dots, n - 1$ belong to the active heap.

Changing a Priority

For a max-heap:

- **Increase-key**: the key may exceed its parent; sift upward.
- **Decrease-key**: the key may become smaller than a child; sift downward.

$$\text{Repair time} = O(\log N)$$

Locating the Element

This bound assumes the element's current index is known. Finding an arbitrary element without an index map can take $O(N)$ time.

Priority Queue Complexity

Operation	Binary Heap Cost
Peek maximum/minimum	$\Theta(1)$
Insert	$O(\log N)$
Extract maximum/minimum	$O(\log N)$
Change key at a known index	$O(\log N)$
Search for an arbitrary key	$O(N)$ worst case
Build heap from an array	$\Theta(N)$
Storage for N elements	$\Theta(N)$

Insertion assumes available capacity. Dynamic-array resizing can make an individual insertion more expensive.

Heap Construction

Given an arbitrary array, construct a heap containing the same keys.

Two approaches:

- ① Insert the keys one at a time into an empty heap.
- ② Build the heap bottom-up using sift-down.

Difference

Repeated insertion: $O(N \log N)$ worst-case time.

Bottom-up construction: $\Theta(N)$ time.

Construction by Repeated Insertion

Start with an empty max-heap and insert:

4, 10, 3, 5, 1.

Inserted key	Heap array
4	[4]
10	[10, 4]
3	[10, 4, 3]
5	[10, 5, 3, 4]
1	[10, 5, 3, 4, 1]

$$\sum_{k=1}^N O(\log k) = O(N \log N).$$

Bottom-Up Heap Construction

- Every leaf is already a one-node heap.
- Start from the last internal node.
- Process internal nodes in decreasing index order.
- Apply sift-down at each node.

$$i = \lfloor N/2 \rfloor - 1, \dots, 1, 0$$

Why This Order Works

When index i is processed, its child subtrees have already been converted into heaps.

Build-Max-Heap Algorithm

```
1 BUILD-MAX-HEAP(A, n)
2   for i = floor(n/2) - 1 downto 0
3     SIFT-DOWN(A, n, i)
```

Example

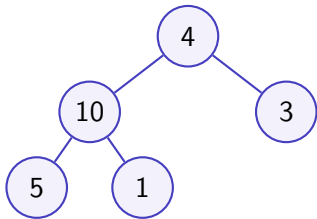
For $n = 5$:

$$\lfloor 5/2 \rfloor - 1 = 1.$$

Process indices 1 and 0. Indices 2, 3, 4 are leaves.

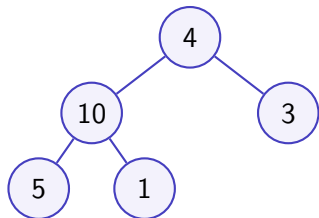
Build-Max-Heap: Initial Array

$A = [4, 10, 3, 5, 1]$



- The shape is complete.
- Root 4 violates max-heap order.
- Leaves 3, 5, 1 already form heaps.
- Start at index 1, containing 10.

Build-Max-Heap: Process Index 1



At index 1:

$$A[1] = 10$$

Its children are:

$$A[3] = 5, \quad A[4] = 1.$$

Since

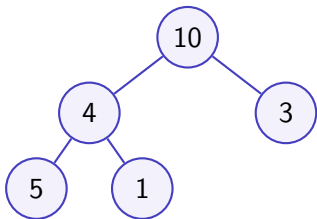
$$10 \geq 5, \quad 10 \geq 1,$$

no swap is needed.

$$A = [4, 10, 3, 5, 1]$$

Build-Max-Heap: Process Index 0

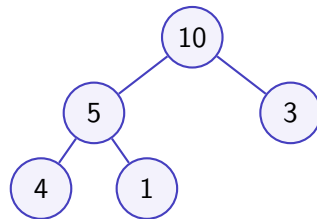
Swap 4 with 10



[10, 4, 3, 5, 1]

The entire array now satisfies max-heap order.

Continue: swap 4 with 5



[10, 5, 3, 4, 1]

Why is Build-Heap Linear?

A node of height h requires at most $O(h)$ sift-down work.

Most nodes are close to the leaves:

- About half the nodes are leaves.
- About one quarter have height 1.
- Only a few nodes have large height.

$$T(N) \leq cN \sum_{h=0}^{\lfloor \log_2 N \rfloor} \frac{h}{2^h} = O(N)$$

because

$$\sum_{h=0}^{\infty} \frac{h}{2^h} = 2.$$

The bound uses at most $N/2^h$ nodes of height h , a sufficient upper bound.

Build-Heap Complexity: Key Distinction

Loose Upper Bound

There are $O(N)$ calls, each costing at most $O(\log N)$:

$$O(N \log N).$$

This is valid but not tight.

Tight Bound

Each call costs at most the height of its node. Summing these different costs gives:

$$\boxed{\Theta(N)}.$$

Bottom-up construction with iterative sift-down uses $O(1)$ auxiliary space.

What is Heap Sort?

Definition

Heap sort uses a binary heap to repeatedly place an extreme element into its final sorted position.

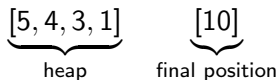
For **ascending order**:

- ① Build a max-heap.
- ② Swap the root with the last active element.
- ③ Shrink the active heap.
- ④ Restore heap order at the root.
- ⑤ Repeat.

Heap Sort: Active Heap and Sorted Suffix



Active max-heap Sorted suffix



Important

Sift-down must use the reduced heap size. It must not touch the sorted suffix.

Heap Sort: Loop Invariant

Before each extraction:

- The active prefix is a max-heap.
- The suffix is sorted in ascending order.
- Every suffix element is at least as large as every element in the active heap.

Progress

The root is the largest active key. Moving it to the end of the active prefix extends the sorted suffix by one position.

Heap Sort Algorithm

```
1 HEAPSORT(A, n)
2   BUILD-MAX-HEAP(A, n)
3
4   for end = n - 1 downto 1
5       SWAP(A[0], A[end])
6       SIFT-DOWN(A, end, 0)
```

Why Pass end?

After the swap, index `end` is sorted. The active heap consists of indices $0, \dots, \text{end} - 1$.

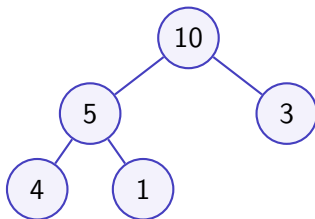
Heap Sort: Worked Example

Input:

[4, 10, 3, 5, 1]

After bottom-up construction:

[10, 5, 3, 4, 1]



The maximum key 10 is ready to move to the final position.

Heap Sort: Pass 1

Swap the root with the last active key:

$$[10, 5, 3, 4, 1] \longrightarrow [1, 5, 3, 4 \mid 10].$$

Restore the heap on the first four positions:

$$[1, 5, 3, 4 \mid 10]$$

$$\longrightarrow [5, 1, 3, 4 \mid 10]$$

$$\longrightarrow \boxed{[5, 4, 3, 1 \mid 10]}.$$

Result

10 is sorted. The active heap size is now 4.

Heap Sort: Passes 2 and 3

Pass 2

Swap 5 with 1:

[1, 4, 3 | 5, 10].

Sift down:

[4, 1, 3 | 5, 10].

Pass 3

Swap 4 with 3:

[3, 1 | 4, 5, 10].

The remaining two-node heap already satisfies heap order.

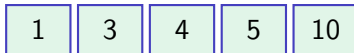
Heap Sort: Final Pass

Swap the remaining root 3 with 1:

$$[3, 1 \mid 4, 5, 10] \longrightarrow [1 \mid 3, 4, 5, 10].$$

A one-node heap needs no repair.

[1, 3, 4, 5, 10]



Heap Sort: Complete Trace

Stage	Active heap	Sorted suffix
Build heap	10, 5, 3, 4, 1	Empty
Pass 1	5, 4, 3, 1	10
Pass 2	4, 1, 3	5, 10
Pass 3	3, 1	4, 5, 10
Pass 4	1	3, 4, 5, 10

Final array: 1, 3, 4, 5, 10

Heap Sort Complexity

Heap construction:

$$\Theta(N)$$

At most $N - 1$ extraction passes:

$$\sum_{k=2}^N O(\log k) = O(N \log N)$$

Therefore:

$$\text{Worst-case time} = \Theta(N \log N)$$

$$\text{Auxiliary space} = O(1)$$

The space bound assumes iterative sift-down. Recursive Max-heapify uses $O(\log N)$ call-stack space.

Properties of Heap Sort

- Comparison-based sorting algorithm.
- In-place with iterative sift-down.
- Worst-case running time: $\Theta(N \log N)$.
- Does not require a separate output array.
- Standard heap sort is **not stable**.

Order of Output

A max-heap produces ascending order in this in-place algorithm. A min-heap produces descending order.

Why is Heap Sort Not Stable?

Suppose equal keys retain labels showing their original order:

$$[2_a, 2_b, 1].$$

A valid max-heap already exists.

After swapping the root with the last position and repairing:

$$[2_b, 1 \mid 2_a].$$

The final swap gives:

$$\boxed{[1, 2_b, 2_a]}.$$

Observation

The equal keys 2_a and 2_b have reversed their order. Therefore standard heap sort is not stable.

C Implementation: Swap and Sift Down

```
1 void swap(int *a, int *b)
2 {
3     int temp = *a;
4     *a = *b;
5     *b = temp;
6 }
7
8 void siftDown(int A[], int n, int i)
9 {
10     while (i < n / 2) {
11         int child = 2 * i + 1;
12
13         if (child + 1 < n && A[child + 1] > A[child])
14             child++;
15
16         if (A[i] >= A[child])
17             break;
18     }
```

C Implementation: Build Heap and Heap Sort

```
1 void buildMaxHeap(int A[], int n)
2 {
3     for (int i = n / 2 - 1; i >= 0; i--)
4         siftDown(A, n, i);
5 }
6
7 void heapSort(int A[], int n)
8 {
9     buildMaxHeap(A, n);
10
11     for (int end = n - 1; end > 0; end--) {
12         swap(&A[0], &A[end]);
13         siftDown(A, end, 0);
14     }
15 }
```

Uses the preceding swap and siftDown functions. Array lengths are nonnegative int values.

C Implementation: Example Driver

```
1 #include <stdio.h>
2
3 /* Place the preceding functions before main. */
4
5 int main(void)
6 {
7     int A[] = {4, 10, 3, 5, 1};
8     int n = (int)(sizeof A / sizeof A[0]);
9
10    heapSort(A, n);
11
12    for (int i = 0; i < n; i++)
13        printf("%d ", A[i]);
14
15    printf("\n");
16    return 0;
17 }
```

Priority Queue: Implementation Choices

Representation	Insert	Peek-Max	Extract-Max
Unsorted array	$O(1)$	$O(N)$	$O(N)$
Sorted array	$O(N)$	$O(1)$	$O(1)$
Binary max-heap	$O(\log N)$	$O(1)$	$O(\log N)$

Assumes available array capacity. In the sorted-array representation, the maximum is at the removable end.

Applications of Priority Queues

- Selecting the next task by priority.
- Processing events by earliest scheduled time.
- Dijkstra's shortest-path algorithm with a min-queue.
- Prim's minimum spanning tree algorithm.
- Huffman coding using repeated minimum extraction.
- Maintaining the largest or smallest k elements.

Choice of Heap

Use a min-heap when the smallest key should be processed first, and a max-heap when the largest should be first.

Practice Question

Given:

$$A = [7, 2, 9, 4, 1, 6],$$

perform the following:

- 1 Construct a max-heap using bottom-up construction.
- 2 Insert key 10 into the resulting heap.
- 3 Extract the maximum from that updated heap.
- 4 State the heap array after each operation.
- 5 Give the worst-case cost of each operation.

Practice Question: Answer

Bottom-Up Construction

Process indices 2, 1, 0:

$$[7, 2, 9, 4, 1, 6] \rightarrow [7, 4, 9, 2, 1, 6] \rightarrow [9, 4, 7, 2, 1, 6].$$

Insert 10

Append and sift upward:

$$[9, 4, 7, 2, 1, 6, 10] \rightarrow [9, 4, 10, 2, 1, 6, 7] \rightarrow [10, 4, 9, 2, 1, 6, 7].$$

Extract-Max

Remove 10 and repair:

$$[7, 4, 9, 2, 1, 6] \rightarrow [9, 4, 7, 2, 1, 6].$$

Practice Question: Complexity

Operation	Worst-case time
Bottom-up construction	$\Theta(N)$
Insert 10	$O(\log N)$
Extract maximum	$O(\log N)$

Multiple Valid Heaps

The same set of keys may have several valid heap layouts. The exact result depends on construction and tie-breaking choices.

Common Mistakes

- Confusing a heap with a BST.
- Forgetting the complete-tree shape requirement.
- Mixing zero-based and one-based index formulas.
- Swapping with the smaller child in a max-heap.
- Starting bottom-up construction from the root.
- Including the sorted suffix during heap-sort repair.
- Claiming heap construction requires $\Theta(N \log N)$.
- Assuming heap sort is stable.

Quick Revision

Concept	Key Point
Binary heap	Complete tree plus heap order
Max-heap root	Maximum key
Min-heap root	Minimum key
Insertion	Append, then sift up
Extract-max	Replace root, then sift down
Build-heap	Process internal nodes bottom-up
Heap construction	$\Theta(N)$
Heap sort	$\Theta(N \log N)$ worst-case time
Iterative heap-sort space	$O(1)$ auxiliary space

Reference

Textbook

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest and Clifford Stein.

Introduction to Algorithms, 3rd edition, MIT Press, 2009.

- Chapter 6: Heapsort.
- Section 6.1: Heaps.
- Section 6.2: Maintaining the heap property.
- Section 6.3: Building a heap.
- Section 6.4: The heapsort algorithm.
- Section 6.5: Priority queues.

Thank You

Questions?